Let’s start with the similarities to testing the forwarding of a message from one actor to another, as in the example in the previous section. This is similar to sync testing, so let’s test the same actor:

```
object SimplifiedManager {

  sealed trait Command
  final case class Forward(message: String, sendTo: ActorRef[String])
      extends Command

    def apply(): Behaviors.Receive[Command] =
      Behaviors.receive { (context, message) =>
        message match {
              ...
          case Forward(text, sendTo) =>
            sendTo ! text
            Behaviors.same
}
```

The test is the same when you test the probe—that is, `probe.expectMessage`—but there are a few differences:

- To create an instance of the actor under test, you use `testKit.spawn`.
- To create the probe, you use `testKit.createTestProbe`.
- The `ActorTestKit` is created only once for the entire spec, and you need to shut down its `ActorSystem` after all the tests are complete. To do this, you add the `BeforeAndAfterAll` trait and override the `afterAll` method. It makes sense to have only one `ActorTestKit` and shut it down at the end because creating and shutting down an `ActorSystem` takes time, and you don’t want to do that for every test in your suite.

This listing shows how everything plays together.